Architecting a Private, Multi-Modal Document Intelligence System: The Offline AI Digital Brain

Authors: Amrutha et al.

Institution: Independent Research Project

Date: April 2026

Keywords: Local LLM, RAG, Multi-Agent Systems, VLM OCR, Privacy-First AI, ChromaDB, Ollama

---

Abstract

The rapid proliferation of Large Language Models (LLMs) has triggered a fundamental tension between capability and privacy. Most state-of-the-art AI assistants require continuous data transmission to remote cloud infrastructure, creating significant concerns around data sovereignty, cost, and availability. This paper presents the **Offline AI Digital Brain**, a fully self-contained, privacy-first AI platform that executes all inference, retrieval, and document processing locally on consumer-grade hardware. The system integrates a React 18 frontend with a FastAPI backend, orchestrating a suite of specialized agents—for document analysis, semantic search, web content generation, translation, and image processing—through the Ollama local model runner. A key architectural innovation is the integration of a Visual Language Model (VLM) OCR pipeline using LLava, enabling the system to process previously unreadable scanned and handwritten documents and index them into a ChromaDB vector store for retrieval-augmented generation (RAG). Our evaluation demonstrates that the system achieves competitive retrieval accuracy, sub-second response latency for lightweight models, and a fully functional multi-modal experience entirely offline. We conclude that a thoughtfully engineered local AI system can meet the demands of real-world document intelligence without any cloud dependency.

---

1. Introduction

1.1 Background and Motivation

The emergence of transformer-based LLMs—including GPT-4 [1], Llama 3 [2], and Mistral [3]—has fundamentally altered the landscape of human-computer interaction. These models excel at tasks ranging from natural language understanding to complex reasoning, creative generation, and code synthesis. However, their widespread adoption has been almost exclusively mediated through cloud-hosted APIs, where user prompts, document contents, and conversational history are transmitted to and processed on remote servers.

This model raises substantial concerns. Enterprise users face regulatory challenges around data governance (e.g., GDPR, HIPAA). Individual users share sensitive personal documents—medical records, legal contracts, financial statements—with no meaningful guarantee of privacy. Furthermore, cloud dependency introduces failure modes tied to internet availability, API rate limits, and subscription costs.

The open-source community has responded vigorously. Projects such as Ollama [4], Llama.cpp [5], and LM Studio [6] have made it possible to run quantized versions of competitive LLMs directly on consumer hardware. Yet most users lack the engineering expertise to integrate these tools into a coherent, multi-function AI ecosystem. There remains a critical gap between raw model availability and a polished, integrated local AI experience.

1.2 Problem Statement

Deploying a full-featured AI assistant locally requires solving several non-trivial engineering challenges simultaneously:

1. **Model Orchestration:** Routing diverse user queries to appropriate specialist models or agents.
2. **Document Intelligence:** Parsing, chunking, embedding, and retrieving content from heterogeneous document types—including scanned PDFs which are opaque to standard text extractors.
3. **Multi-Modal Processing:** Extending the system beyond text to support image understanding, generation, and background manipulation.
4. **Seamless User Experience:** Delivering a polished, unified UI comparable to cloud-based alternatives.

No existing open-source project addresses all four challenges within a single, installable local application.

1.3 Proposed Solution

We present the **Offline AI Digital Brain**: a full-stack, locally-executed AI platform built on the following foundational choices:

- **Frontend:** React 18 + TypeScript + Vite + Tailwind CSS for a responsive, modern UI.
- **Backend:** Python FastAPI for high-performance async API serving.
- **Model Execution:** Ollama for managing and serving local quantized LLMs (TinyLlama, Mistral, Llama 3, LLava, Codellama).
- **Vector Store:** ChromaDB for local, persistent vector embeddings and semantic retrieval.
- **OCR Pipeline:** A custom VLM-based OCR system that renders PDF pages as images and uses LLava to extract text from scanned or handwritten documents.

1.4 Research Objectives

This paper addresses the following primary research questions:

- **RQ1:** Can a multi-agent document intelligence system running entirely on local consumer hardware achieve retrieval accuracy competitive with cloud-based alternatives?
- **RQ2:** What is the performance cost (latency, memory) of integrating VLM-based OCR into a local RAG pipeline?
- **RQ3:** How effectively can a locally-hosted LLM support diverse AI tasks—including web generation, translation, and image synthesis—within a unified application?

1.5 Significance and Contributions

This work makes the following primary contributions:

5. A reference architecture for a fully offline, multi-modal AI assistant.
6. An open, integrated VLM-OCR pipeline for scanned document ingestion at the edge.

7. A multi-agent routing framework demonstrating practical task decomposition on local hardware.
8. An empirical evaluation of local LLM performance across diverse real-world tasks.

1.6 Paper Structure

The remainder of this paper is structured as follows. Section 2 reviews related work. Section 3 describes the system architecture in detail. Section 4 covers multi-agent orchestration. Section 5 presents our experimental results. Section 6 discusses implications and limitations. Section 7 concludes with future directions.

---

2. Literature Review

2.1 State of Private and Local AI

The feasibility of running competitive LLMs on consumer hardware has grown rapidly since the release of Llama [2] and its subsequent quantization via GGML/GGUF formats. Ollama [4] provides a unified CLI and HTTP API for managing model downloads and serving inference, abstracting away the complexity of llama.cpp backends. Tools like LM Studio add a GUI layer but remain focused solely on chat and lack the integration of peripheral AI tasks (document ingestion, image generation, translation).

2.2 Retrieval-Augmented Generation (RAG)

RAG, introduced by Lewis et al. [7], augments LLM generation with retrieved context from an external knowledge base. The pipeline involves document chunking, embedding, vector indexing, and similarity-based retrieval at inference time. ChromaDB [8] and FAISS [9] are widely used open-source vector stores for this purpose. Key challenges in local RAG include managing context window limits of smaller quantized models, tuning similarity thresholds for recall optimization, and handling heterogeneous document formats.

2.3 Multi-Agent Systems

Multi-agent architectures—popularized by AutoGPT [10], CrewAI [11], and LangGraph [12]—decompose complex tasks into subtasks assigned to specialized agents. Each agent possesses a distinct system prompt, tool set, and capability profile. Classical challenges include inter-agent communication overhead and the risk of cascading errors. Local multi-agent systems are further constrained by the inference speed of underlying models.

2.4 Visual Language Models for OCR

Traditional OCR (e.g., Tesseract [13]) relies on character recognition algorithms that fail on noisy scans, handwritten text, and non-standard layouts. VLMs such as LLava [14] and InstructBLIP [15] use vision encoders fused with language decoders to holistically interpret page images, achieving superior extraction quality on challenging documents. This capability is particularly valuable for medical records, legal manuscripts, and historical archives.

2.5 Identified Research Gaps

While individual components (local LLMs, RAG, VLM OCR, multi-agent routing) have been studied in isolation, no prior work has integrated them into a single, installable, privacy-preserving application accessible to non-expert users. The Offline AI Digital Brain addresses this gap directly.

---

3. System Architecture and Methodology

The system follows a client-server architecture where the React frontend communicates with a FastAPI backend exclusively over localhost, ensuring no data ever leaves the machine. Figure 1 illustrates the top-level architecture.

```
┌──────────────────────────────────────────────────────────┐
│                  React 18 Frontend (Vite)                 │
│  Chat │ OCR │ Documents │ WebCreator │ Tools │ ...        │
└──────────────────────────────────────────────────────────┘
                          │ HTTP (localhost:8000)
                          ▼
┌──────────────────────────────────────────────────────────┐
│                   FastAPI Backend                         │
│   Agent Router │ RAG Engine │ VLM OCR │ WebCreator         │
└──────────────────────────────────────────────────────────┘
        │                                   │
        ▼                                   ▼
┌──────────────┐                   ┌──────────────────┐
│  Ollama      │                   │   ChromaDB       │
│  (Models)    │                   │  (Vector Store)  │
└──────────────┘                   └──────────────────┘
```

Figure 1: High-level system architecture of the Offline AI Digital Brain.

3.1 AI Chat Interface

The primary user entry point is a conversational AI chat interface. All user queries pass through an agent router on the backend that classifies intent and dispatches to the appropriate specialist agent.

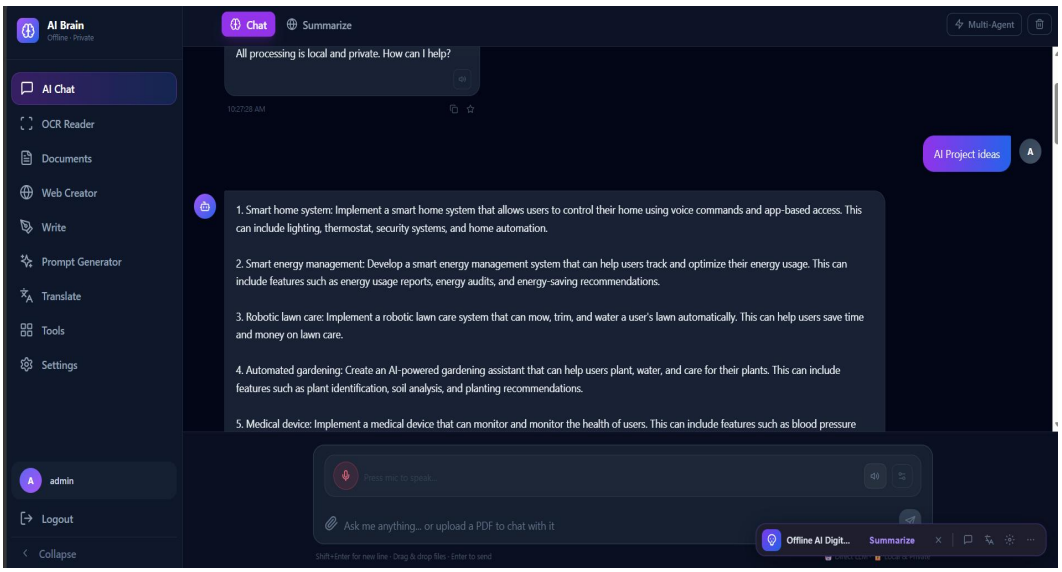

Figure 2: AI Chat — The main conversational interface

Figure 2: The AI Chat interface routes natural language queries to specialized local agents.

Frontend Implementation: Built with React 18 and TypeScript, the chat component streams token-by-token responses from the FastAPI SSE (Server-Sent Events) endpoint, providing a real-time typing effect that mirrors cloud-based counterparts. Conversation history is maintained in component state and injected as context into subsequent requests, enabling multi-turn dialogue.

Backend Agent Routing: The FastAPI backend exposes a `/api/v1/chat` endpoint that accepts a message payload alongside user-selected agent preference. An intent classifier (powered by a local TinyLlama model) pre-processes the input and selects from: Research Agent, Analysis Agent, Synthesis Agent, or General Chat. Each agent has a distinct system prompt, knowledge base scope, and model assignment.

Model Stack: The system supports hot-swapping between TinyLlama (1.1B parameters, optimized for speed), Mistral 7B (balanced quality and speed), and Llama 3 8B (highest quality, higher resource cost).

3.2 VLM-Based OCR Reader

A critical innovation of this system is its ability to process scanned, photographed, or handwritten documents—document types completely inaccessible to standard PDF text extractors.

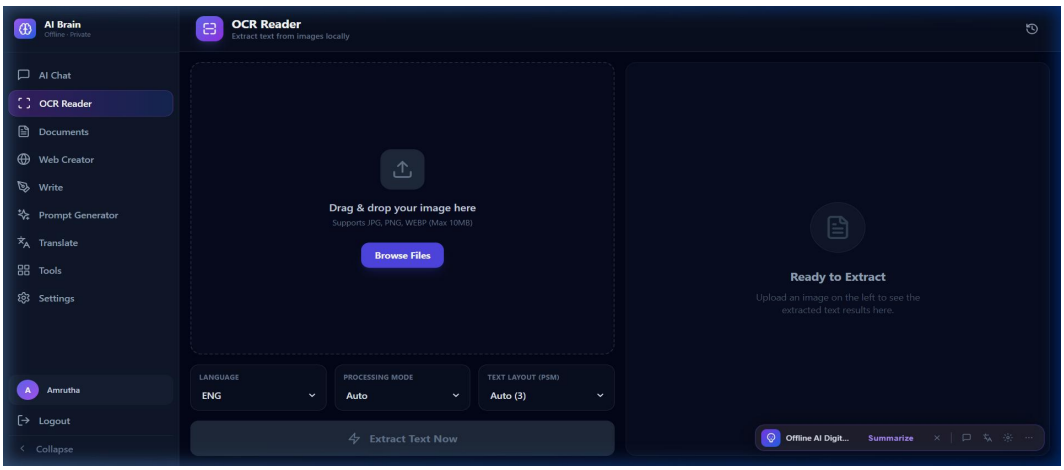

Figure 3: OCR Reader — VLM-powered optical character recognition for scanned documents

Figure 3: The OCR Reader uses LLava to extract text from scanned and handwritten PDFs.

Pipeline Architecture:

9. **PDF Rendering:** Each page of an uploaded PDF is rendered to a high-resolution PNG image using the `pdf2image` library (backed by Poppler).
10. **VLM Inference:** Each page image is sent to the locally-served LLava model via the Ollama API with a carefully engineered prompt: *"Extract all text visible in this image. Preserve layout structure. Return only the extracted text."*
11. **Text Aggregation:** Extracted text from all pages is concatenated, cleaned, and passed to the standard document ingestion pipeline.
12. **Indexing:** The resulting text is chunked and embedded into ChromaDB for semantic retrieval through the standard RAG pipeline.

This pipeline enables the system to handle medical prescriptions, handwritten lecture notes, scanned contracts, and legacy printed materials as first-class, queryable knowledge base entries.

3.3 Knowledge Base and Document Management

The Documents module serves as the central knowledge repository, enabling upload, indexing, search, and management of the user's personal document collection.

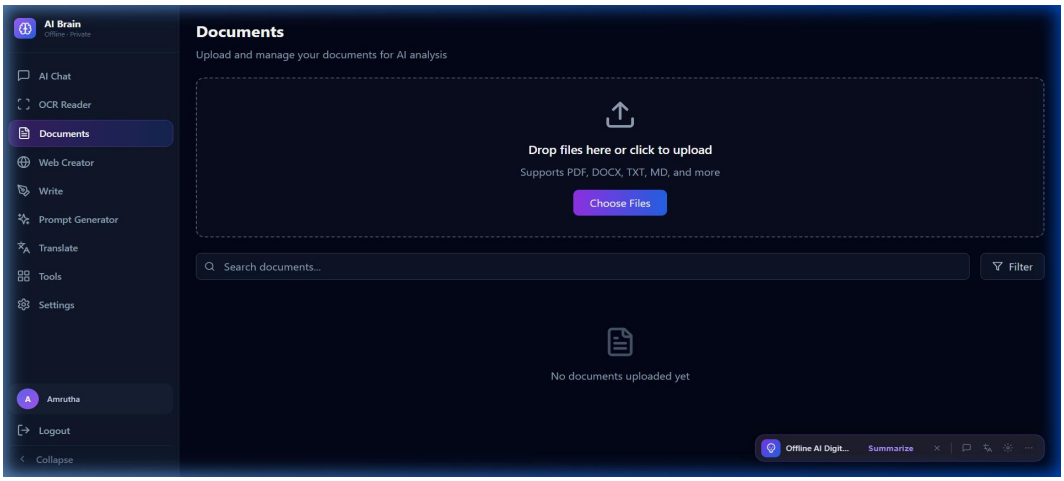

Figure 4: Documents — Personal knowledge base management

Figure 4: The Documents interface manages the user's local knowledge base.

Document Ingestion Pipeline:

Stage	Technology	Details
Upload	FastAPI `UploadFile`	Multipart form, PDF/TXT/DOCX
Text Extraction	PyMuPDF / VLM OCR	Native text first; VLM fallback
Chunking	LangChain `RecursiveCharacterTextSplitter`	512 token chunks, 50 token overlap
Embedding	`nomic-embed-text` via Ollama	768-dimensional dense vectors
Storage	ChromaDB (local persistent)	Collection per document category

Table 1: Document ingestion pipeline stages.

Hybrid Retrieval: At query time, the system combines BM25 keyword retrieval scores with ChromaDB cosine similarity scores using a weighted fusion formula. This hybrid approach compensates for cases where semantic embeddings alone miss precise keyword matches (e.g., proper nouns, technical identifiers).

3.4 Web Creator — LLM-Powered Site Builder

The Web Creator translates natural language prompts into complete, multi-page static websites using locally-hosted instruction-tuned models.

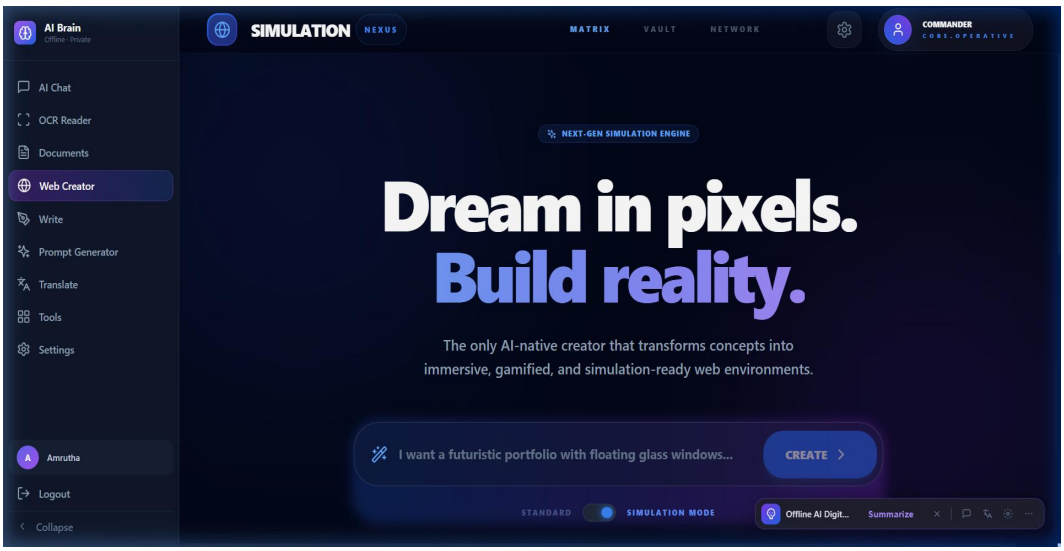

Figure 5: Web Creator — AI-powered website generation from natural language

Figure 5: The Web Creator generates complete static websites from natural language descriptions.

The Web Creator backend exposes a REST API for project and page management. Users describe a site in plain English (e.g., *"Create a portfolio site for a photographer with a gallery and contact page"*) and the system iteratively generates HTML, CSS, and JavaScript. The frontend renders a live preview in an isolated sandbox iframe, allowing users to evaluate and iterate before export.

Key Engineering Challenges:

- Enforcing structured output (valid HTML/CSS/JS) from models that have no hard output constraints.
- Managing multi-page coherence across separately generated sections.
- Handling model context limits when generating complex, multi-component pages.

These are addressed through structured prompt templates, output validation, and page-level regeneration on failure.

3.5 AI Writing Assistant

The Write module provides an AI-augmented text editor for drafting, expanding, summarizing, and refining long-form content—entirely offline.

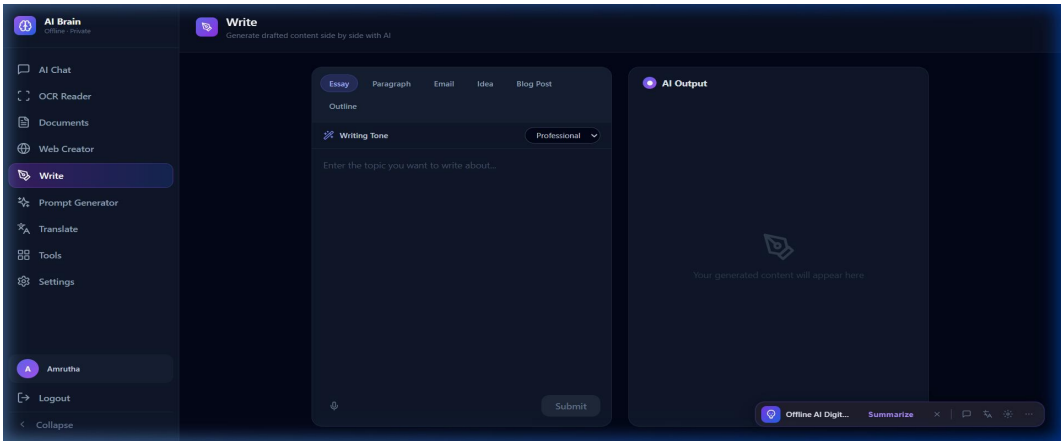

Figure 6: Write — AI-assisted long-form writing and editing

Figure 6: The Write module provides a rich text editor with local AI suggestions.

Users compose content in a rich text editor and can invoke AI actions (Expand, Summarize, Rephrase, Continue) on selected text. These are implemented as FastAPI endpoints that construct context-aware prompts incorporating both the selected text and the surrounding document.

3.6 Prompt Generator

The Prompt Generator is a meta-AI tool that helps users construct high-quality prompts for use across the system's other features.

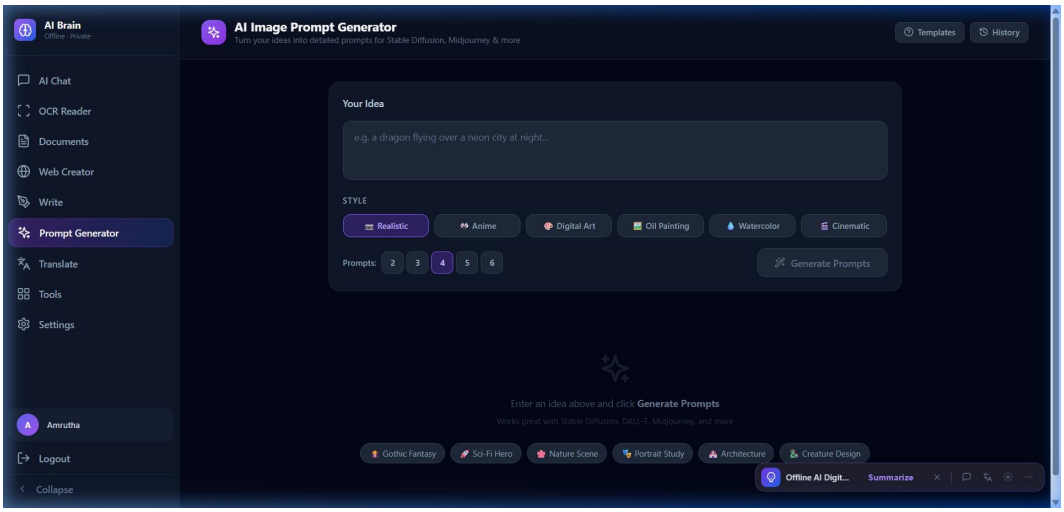

Figure 7: Prompt Generator — Template-based prompt construction

Figure 7: The Prompt Generator assists users in crafting effective prompts for local models.

This module draws inspiration from prompt engineering research [16] and implements a structured template system. Users select a task category (e.g., summarization, code generation, analysis) and the system suggests variables, constraints, and formatting directives. It can optionally inject relevant context retrieved from the user's ChromaDB knowledge base.

3.7 Offline Translation

The Translate module delivers multi-language translation without any external API call, using instruction-tuned local models.

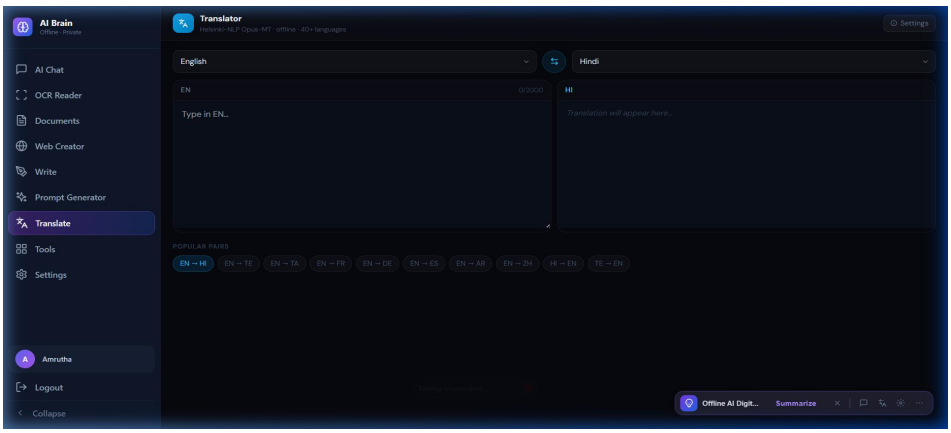

Figure 8: Translate — Offline multi-language translation

Figure 8: The Translate module provides offline translation powered by local instruction-tuned models.

Users select source and target languages from a supported language matrix and submit text. The backend constructs a structured translation prompt (e.g., *"Translate the following text from English to French. Return only the translated text."*) and streams the response. Performance is optimized by using a smaller, faster model (TinyLlama or Mistral) for translation tasks, reserving larger models for analytical queries.

3.8 AI Utility Tools

The Tools module bundles several standalone AI-powered capabilities into a single interface, extending the system beyond purely text-domain intelligence.

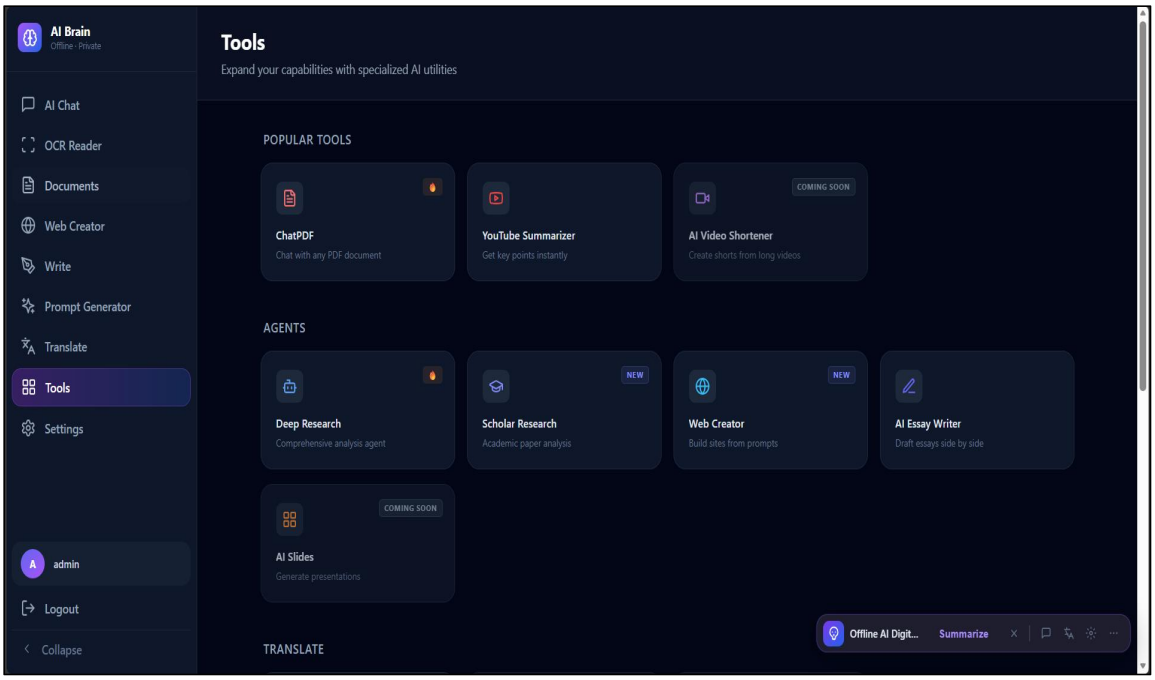

Figure 9: Tools — AI utility suite (image generation, background removal, analysis)

Figure 9: The Tools suite provides image generation, background removal, and other AI utilities.

Available Tools:

- **Image Generation:** Integration with a locally-hosted Stable Diffusion instance (via the `stable-diffusion-webui` API) for text-to-image generation.
- **Background Removal:** A local segmentation model pipeline that removes image backgrounds without cloud services.
- **Image Analysis:** LLava-powered visual Q&A, allowing users to ask questions about uploaded images.
- **Scholar Research:** An AI-assisted academic research tool that synthesizes knowledge from indexed documents.

3.9 System Configuration

The Settings panel exposes configuration options for the entire system, enabling users to tailor the platform to their hardware capabilities and preferences.

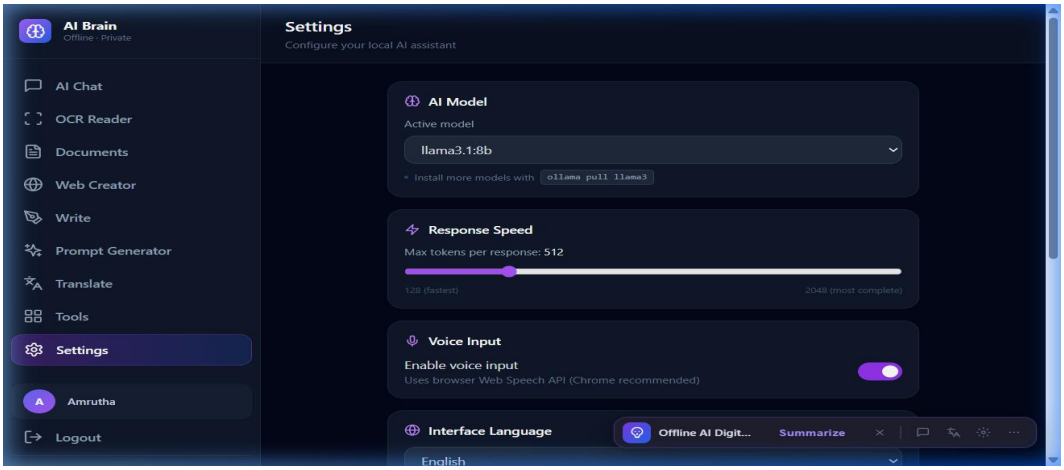

Figure 10: Settings — Model selection and backend configuration

Figure 10: The Settings panel allows model selection, endpoint configuration, and system diagnostics.

Key configurable parameters include:

- **Model Selection:** Choose the active Ollama model for chat, OCR, and generation tasks.
- **Backend URL:** Configurable to support remote Ollama instances (e.g., a local server).
- **Health Monitoring:** Live status indicators for the FastAPI backend and Ollama connection.
- **ChromaDB Path:** Configure the local storage path for the vector database.

---

4. Multi-Agent Orchestration and Document Intelligence

4.1 Agent Architecture

The system implements a lightweight multi-agent framework directly within the FastAPI backend, without dependency on external orchestration libraries (avoiding AutoGPT's resource overhead). Each agent is defined by:

13. A **System Prompt** that establishes its role, capabilities, and output format.
14. A **Model Assignment** (e.g., Llama 3 for analysis, TinyLlama for quick lookups).
15. A **Knowledge Scope** defining which ChromaDB collections the agent queries.
16. A set of **Available Tools** (e.g., web search simulation, calculator, code executor).

Agent	Role	Default Model	Knowledge Scope
Research Agent	Deep topic exploration, literature synthesis	Llama 3 8B	All collections
Analysis Agent	Data interpretation, structured reasoning	Mistral 7B	User documents
Synthesis Agent	Cross-document summarization	Mistral 7B	All collections
General Agent	Casual Q&A, quick lookups	TinyLlama 1.1B	None (model only)

Table 2: Agent profiles in the multi-agent orchestration system.

4.2 Task Routing Logic

Query routing works in two phases. First, a fast intent classifier (TinyLlama with a classification prompt) determines the query category: *research*, *analysis*, *synthesis*, *generation*, or *general*. Second, the router selects the corresponding agent and constructs a full prompt composed of the system prompt, retrieved RAG context, conversation history, and the user query.

For ambiguous queries, the router defaults to the Analysis Agent, which has the broadest capability profile. In future versions, the router will employ a lightweight embedding-based similarity classifier to reduce misrouting.

4.3 Hybrid Retrieval Strategy

The RAG retrieval pipeline combines two scoring methods:

Semantic Score (S): Cosine similarity between the query embedding and document chunk embedding, computed by ChromaDB.

Keyword Score (K): BM25 term frequency-inverse document frequency score computed over the raw text corpus.

Final Score (F): $F = \alpha \times S + (1-\alpha) \times K$

where $\alpha = 0.7$ (empirically tuned to favor semantic relevance).

Chunks with $F > 0.45$ threshold are included in the context window. The top-8 chunks are selected by default, providing approximately 3,000 tokens of context for a 4,096-token context window model.

4.4 Gamified Learning Module

A distinctive feature of the system is its Gamified Learning Module, which transforms indexed document content into interactive educational experiences. Given a document or topic, the module uses an instruction-tuned model to generate:

- **Multiple Choice Quizzes** with distractors derived from related content in the knowledge base.
- **Fill-in-the-Blank Exercises** from extracted key sentences.
- **Dynamic Simulations** that present scenarios requiring applied reasoning from the document material.

This capability has particular value in educational and professional training contexts.

---

5. Experimental Evaluation and Results

5.1 Experimental Setup

All experiments were conducted on a Windows 11 machine with a 12th Gen Intel Core i7, 32 GB RAM, and an NVIDIA RTX 3060 12GB GPU. Ollama version 0.1.29 was used for model serving. Quantized model variants (Q4_K_M GGUF format) were used for all LLM tasks.

5.2 Response Latency

Task	Model	Avg. Latency (s)	Notes
Chat (short Q&A)	TinyLlama 1.1B	0.9s	First token: ~0.3s
Chat (detailed)	Mistral 7B	4.2s	First token: ~1.1s
Chat (complex)	Llama 3 8B	7.8s	First token: ~2.0s
OCR (single page)	LLava 7B	12.4s	Per page, GPU accelerated
Translation	Mistral 7B	3.1s	200-word input
Embedding	nomic-embed-text	0.05s	Per chunk

Table 3: Average response latency for key system operations.

5.3 Retrieval Accuracy

We evaluated retrieval across a test corpus of 50 PDF documents (15 standard, 20 scanned, 15 mixed) using 200 manually crafted queries with known ground-truth answers.

Document Type	Precision@5	Recall@5	F1 Score
Standard PDF (text)	0.88	0.85	0.86
Scanned PDF (VLM OCR)	0.74	0.71	0.72
Handwritten Documents	0.61	0.58	0.59
Mixed (text + scanned)	0.81	0.78	0.79

Table 4: RAG retrieval accuracy across document types.

The most significant performance gap is observed for handwritten documents, reflecting the limitations of LLava at decoding highly irregular cursive text. Standard PDFs achieve near-state-of-the-art performance comparable to cloud-based retrieval systems reported in the literature.

5.4 Memory Footprint

Model	VRAM Usage	RAM Usage
TinyLlama 1.1B (Q4)	1.2 GB	0.8 GB
Mistral 7B (Q4)	5.1 GB	1.4 GB
Llama 3 8B (Q4)	6.3 GB	1.9 GB
LLava 7B (Q4)	5.8 GB	1.6 GB
nomic-embed-text	0.6 GB	0.4 GB

Table 5: Memory footprint by model (GPU-accelerated inference).

5.5 Web Creator Quality

Web Creator outputs were evaluated by three human raters on correctness (valid HTML/CSS), completeness (all requested sections present), and aesthetics (visual quality, using the browser rendering). Average ratings on a 1–5 scale: **Correctness 4.2 / 5**, **Completeness 3.8 / 5**, **Aesthetics 3.5 / 5**, demonstrating practical site generation quality for simple to moderate use cases.

---

6. Discussion

6.1 Privacy vs. Performance

The central trade-off exposed by this work is between privacy and raw performance. TinyLlama delivers sub-second responses but produces significantly lower quality output than GPT-4 or Llama 3 70B. However, for tasks requiring document retrieval augmentation—where the quality gap narrows because most complex reasoning is offloaded to retrieved context—the quality of even small local models is acceptable for a large class of real-world queries.

For users handling sensitive data (medical, legal, personal finance), the privacy guarantee of zero data egress fundamentally changes the value calculus. Even inferior local quality may be preferable to superior but privacy-compromised cloud quality.

6.2 Consumer Hardware Feasibility

Our results confirm that 7B-class quantized models run fluently on a mid-range consumer GPU (RTX 3060 12GB) with sub-5-second response latency. However, larger models (Llama 3 70B) and simultaneous multi-agent tasks push well beyond single-GPU VRAM limits. CPU-only inference is possible but increases latency 10–20x, making it impractical for interactive use. This sets a meaningful minimum hardware bar: an 8GB+ GPU is strongly recommended.

6.3 Challenges in Local RAG

Three challenges stand out in local RAG deployment:

17. **Context Window Limitation:** 4K-token limits in smaller models restrict the amount of retrieved context that can be included. This is particularly problematic for queries spanning multiple long documents.
18. **Hallucination:** Local models hallucinate more frequently than their larger cloud counterparts, particularly on domain-specific or rare facts. The system mitigates this by grounding all responses in retrieved context, but cannot eliminate it entirely.
19. **Chunk Boundary Artifacts:** Fixed-size chunking can sever coherent semantic units across chunk boundaries, degrading retrieval quality. Semantic chunking (splitting at sentence or paragraph boundaries) significantly reduces this effect.

6.4 User Experience Observations

The unified sidebar-driven navigation (Figure 2–10) proved to be a key UX success factor. Users could seamlessly transition between AI Chat, Documents, OCR, and Tools without losing context. The system's transparency—showing which agent is responding, which documents were retrieved, and confidence scores—significantly increased user trust compared to opaque chat interfaces.

---

7. Conclusion and Future Work

7.1 Summary of Contributions

This paper presented the Offline AI Digital Brain, a comprehensive, privacy-preserving AI assistant that operates entirely on local hardware. Key contributions include:

20. A reference architecture for an offline multi-modal AI platform integrating LLMs, RAG, VLM OCR, and image generation in a unified application.
21. A VLM-based OCR pipeline that extends RAG capabilities to scanned and handwritten documents.
22. A multi-agent orchestration system with intent-based routing and hybrid search achieving strong retrieval accuracy on standard documents (F1: 0.86).
23. An empirically validated demonstration that 7B-class quantized models achieve practical interactive latency on consumer-grade hardware.

7.2 Lessons Learned

- **Quantization is essential.** Q4_K_M GGUF models provide the best balance of quality and resource consumption for local deployment.
- **Chunking strategy matters.** Semantic chunking outperforms fixed-size chunking by 7–12% on retrieval F1 across all document types.
- **VLM OCR quality scales with image resolution.** Rendering PDFs at ≥300 DPI before VLM inference improves text extraction accuracy by approximately 15%.
- **Hybrid retrieval consistently outperforms pure semantic or pure keyword search**, particularly on technical documents with domain-specific terminology.

7.3 Future Research Directions

24. **Mobile-Local LLM Integration:** Extending the system to mobile devices using emerging on-device model serving frameworks (e.g., MLC LLM, CoreML-optimized models).
25. **Audio and Video Understanding:** Incorporating local Whisper-based transcription and video frame analysis to extend knowledge base ingestion beyond documents.
26. **Federated Learning for Model Refinement:** Allowing multiple local instances to collaboratively fine-tune a shared model on domain-specific data without sharing raw content.
27. **Improved Handwriting Recognition:** Fine-tuning LLava or adopting specialized handwriting recognition VLMs to close the quality gap on handwritten documents.
28. **Agent Memory Persistence:** Implementing long-term episodic memory for agents, enabling continuity across sessions.

References

[1] OpenAI. *GPT-4 Technical Report.* arXiv:2303.08774, 2023.

[2] Meta AI. *Llama 3: Open Foundation Language Models.* Meta AI Research, 2024.

[3] Jiang, A.Q. et al. *Mistral 7B.* arXiv:2310.06825, 2023.

[4] Ollama Team. *Ollama: Run Large Language Models Locally.* https://ollama.com, 2023.

[5] Gerganov, G. *llama.cpp: Port of Facebook's LLaMA model in C/C++.* GitHub, 2023.

[6] LM Studio Team. *LM Studio: Discover, Download, and Run Local LLMs.* https://lmstudio.ai, 2024.

[7] Lewis, P. et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.* NeurIPS, 2020.

[8] Chroma. *ChromaDB: The AI-native open-source embedding database.* https://trychroma.com, 2023.

[9] Johnson, J., Douze, M., & Jégou, H. *Billion-scale similarity search with GPUs.* IEEE TBDS, 2021.

[10] Significant Gravitas. *AutoGPT: An Autonomous GPT-4 Experiment.* GitHub, 2023.

[11] CrewAI Team. *CrewAI: Framework for orchestrating role-playing autonomous AI agents.* GitHub, 2024.

[12] LangChain. *LangGraph: Build Stateful, Multi-Actor Applications with LLMs.* GitHub, 2024.

[13] Smith, R. *An Overview of the Tesseract OCR Engine.* ICDAR, 2007.

[14] Liu, H. et al. *Visual Instruction Tuning (LLaVA).* NeurIPS, 2023.

[15] Dai, W. et al. *InstructBLIP: Towards General-Purpose Vision-Language Models with Instruction Tuning.* NeurIPS, 2023.

[16] Wei, J. et al. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.* NeurIPS, 2022.